TRƯỜNG ĐẠI HỌC NGÂN HÀNG
THÀNH PHỐ HỒ CHÍ MINH

KHAI PHÁ DỮ LIỆU

CHƯƠNG 6
MỘT SỐ XU HƯỚNG KHAI PHÁ DỮ LIỆU VÀ CÁC CHỦ ĐỀ MỞ RỘNG KHÁC

Bộ môn Khoa học máy tính - Khoa Hệ thống thông tin quản lý



TRƯỜNG ĐẠI HỌC NGÂN HÀNG
THÀNH PHỐ HỒ CHÍ MINH

Bộ môn Khoa học máy tính - Khoa Hệ thống thông tin quản lý

Nội dung

6.1 Xu hướng khai phá dữ liệu

6.1.1 Khai phá dữ liệu tuần tự

6.1.2 Khai phá mạng và đồ thị

6.1.3 Khai phá một số loại dữ liệu khác

6.2 Khai phá dữ liệu trực quan

6.3 **Deep learning**

6.4 **Dimensionality reduction**

6.5 **Feature engineering**

Introducing Deep Learning

- In recent years, so-called "deep learning" approaches to machine learning have had a major impact on speech recognition and computer vision
- Other disciplines, such as natural language processing, are also starting to see benefits
- A critical ingredient is the use of much larger quantities of data than has heretofore been possible
- Recent successes have arisen in settings involving high capacity models—ones with many parameters
- Here, deep learning methods create flexible models that exploit information buried in massive datasets far more effectively than do traditional machine learning techniques using hand-engineered features

Views on machine learning

- One way to view machine learning is in terms of three general approaches:
- Classical machine learning techniques,
  which make predictions directly from a set of features that have been pre-specified by the user;
- Representation learning techniques,
  which transform features into some intermediate representation prior to mapping them to final predictions; and
- Deep learning techniques,
  a form of representation learning that uses multiple transformation steps to create very complex features

The neural network renaissance and deep learning revolution

- The term “renaissance” captures a massive resurgence of interest in neural networks and deep learning techniques
- Many high-profile media (e.g. The New York Times) have documented the striking successes of deep learning techniques on key benchmark problems
- Starting around 2012, impressive results were achieved on long-standing problems in speech recognition and computer vision, and in competitive challenges such as the ImageNet Large Scale Visual Recognition Challenge and the Labeled Faces in the Wild evaluation

GPUs, graphs and tensors

- The easy availability of high-speed computation in the form of graphics processing units has been critical to the success of deep learning techniques
- When formulated in matrix-vector form, computation can be accelerated using optimized graphics libraries and hardware
- This is why we will study backpropagation in matrix-vector form
  - ✓Readers unfamiliar with manipulating functions that have matrix arguments, and their derivatives are advised to consult Appendix A.1 for a summary of some useful background
- As network models become more complex, some quantities can only be represented using multidimensional arrays of numbers
  - ✓ Such arrays are sometimes referred to as tensors, a generalization of matrices that permit an arbitrary number of indices
- Software for deep learning supporting computation graphs and tensors is therefore invaluable for accelerating the creation of complex network structures and making it easier to learn them

Key developments

- The following developments have played a crucial role in the resurgence of neural network methods:
- the proper evaluation of machine learning methods;
- vastly increased amounts of data;
- deeper and larger network architectures;
- accelerated training using GPU techniques

Mixed National Institute of Standards and Technology (MNIST)

- Is a database and evaluation setup for handwritten digit recognition
- Contains 60,000 training and 10,000 test instances of hand-written digits, encoded as 28×28 pixel grayscale images
- The data is a re-mix of an earlier NIST dataset in which adults generated the training data and high school students generated the test set
- Lets compare the performance of different methods

MNIST

Classifier	Test Error Rate (%)	References
Linear classifier (1-layer neural net)	12.0	LeCun et al. (1998)
K-nearest-neighbors, Euclidean (L2)	5.0	LeCun et al. (1998)
2-Layer neural net, 300 hidden units, mean square error	4.7	LeCun et al. (1998)
Support vector machine, Gaussian kernel	1.4	MNIST Website
Convolutional net, LeNet-5 (no distortions)	**0.95**	LeCun et al. (1998)
Methods using distortions		
Virtual support vector machine, deg-9 polynomial, (2-pixel jittered and deskewing)	0.56	DeCoste and Scholkopf (2002)
Convolutional neural net (elastic distortions)	0.4	Simard, Steinkraus, and Platt (2003)
6-Layer feedforward neural net (on GPU) (elastic distortions)	**0.35**	Ciresan, Meier, Gambardella, and Schmidhuber (2010)
Large/deep convolutional neural net (elastic distortions)	**0.35**	Ciresan, Meier, Masci, Maria Gambardella, and Schmidhuber (2011)
Committee of 35 convolutional networks (elastic distortions)	0.23	Ciresan, Meier, and Schmidhuber (2012)

Losses and regularization

- Logistic regression can be viewed as a simple neural network with no hidden units
- The underlying optimization criterion for predicting i=1,…,N labels yi from features xi with parameters θ consisting of a matrix of weights W and a vector of biases b can be viewed as

$$\sum_{i=1}^{N} -\log p(y_i \mid \mathbf{x}_i; \mathbf{W}, \mathbf{b}) + \lambda \sum_{j=1}^{M} w_j^2 = \sum_{i=1}^{N} L(f_i(\mathbf{x}_i; \theta), y_i) + \lambda R(\theta)$$

- where the first term, $L(f_i(\mathbf{x}_i;\theta), y_i)$, is the negative conditional log-likelihood or loss, and
- the second term, $\lambda R(\theta)$, is a weighted regularizer used to prevent overfitting

Empirical risk minimization

- This formulation as a loss- and regularizer-based objective function gives us the freedom to choose either probabilistic losses or other loss functions
- Using the average loss over the training data, called the empirical risk, leads to the following formulation of the optimization problem: minimize the empirical risk plus a regularization term, i.e.

$$\arg\min_{\theta}\left[\frac{1}{N}\sum_{i=1}^{N}L(f_i(\mathbf{x}_i;\theta),\mathbf{y}_i)+\lambda R(\theta)\right].$$

- Note that the factor N must be accounted for if one relates the regularization weight here to the corresponding parameter derived from a formal probabilistic model for a distribution on parameters

In practice

- In deep learning we are often interested in examining learning curves that show the loss or some other performance metric on a graph as a function of the number of passes that an algorithm has taken over the data.
- It is much easier to compare the average loss over a training set with the average loss over a validation set on the same graph, because dividing by N gives them the same scale.

Common losses for neural networks

- The final output function of a neural network typically has the form $f_k(\mathbf{x})=f_k(a_k(\mathbf{x}))$, where $a_k(\mathbf{x})$ is just one of the elements of vector fuction $\mathbf{a}(\mathbf{x})=\mathbf{W}\mathbf{h}(\mathbf{x})+\mathbf{b}$
- Commonly used output loss functions, output activation functions, and the underlying distributions from which they derive are shown below

Loss Name, $L(f_i(\mathbf{x}_i;\theta),\mathbf{y}_i)=$	**Distribution Name**, $P(f_i(\mathbf{x}_i;\theta),\mathbf{y}_i)=$	**Output Activation Function**, $f_k(a_k(\mathbf{x}))=$
Squared error, $\sum_{k=1}^{K}(f_k(\mathbf{x})-y_k)^2$	Gaussian, $N(\mathbf{y};\mathbf{f}(\mathbf{x};\theta),\mathbf{I})$	$\frac{1}{(1+\exp(-a_k(\mathbf{x})))}$
Cross entropy, $-\sum_{k=1}^{K}[y_k\log f_k(\mathbf{x})+(1-y_k)\log(1-f_k(\mathbf{x}))]$	Bernoulli, $\text{Bern}(\mathbf{y};\mathbf{f}(\mathbf{x};\theta))$	$\frac{1}{(1+\exp(-a_k(\mathbf{x})))}$
Softmax, $-\sum_{k=1}^{K}y_k\log f_k(\mathbf{x})$	Discrete or Categorical, $\text{Cat}(\mathbf{y};\mathbf{f}(\mathbf{x};\theta))$	$\frac{\exp(a_k(\mathbf{x}))}{\sum_{j=1}^{K}\exp(a_j(\mathbf{x}))}$

Deep neural network architectures

- Compose computations performed by many layers
- Denoting the output of hidden layers by h(l)(x), the computation for a network with L hidden layers is:

$$\mathbf{f}(\mathbf{x}) = \mathbf{f}\left[\mathbf{a}^{(L+1)}\left(\mathbf{h}^{(L)}\left(\mathbf{a}^{(L)}\left(\ldots\left(\mathbf{h}^{(2)}\left(\mathbf{a}^{(2)}\left(\mathbf{h}^{(1)}\left(\mathbf{a}^{(1)}(\mathbf{x})\right)\right)\right)\right)\right)\right)\right]$$

- Where pre-activation functions a(l)(x) are typically linear, of the form with matrix W(l) and bias b(l)

$$\mathbf{a}^{(l)}(\mathbf{x}) = \mathbf{W}^{(l)}\mathbf{x} + \mathbf{b}^{(l)}$$

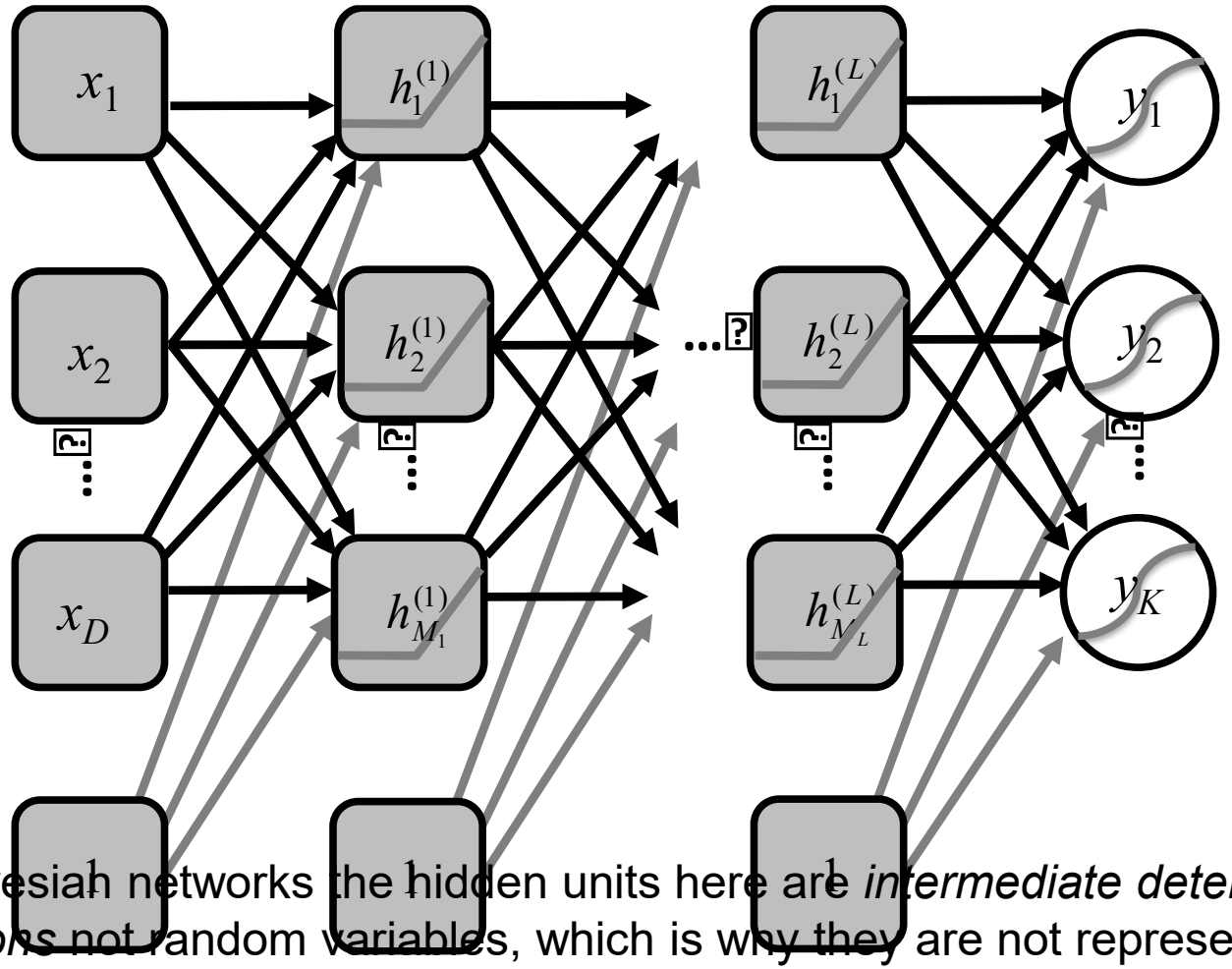
- This formulation can be expressed using a single parameter matrix θ with the trick of defining as x with a 1 appended to the end of the vector; we then have

$$\hat{\mathbf{x}}$$

$$\mathbf{a}^{(l)}(\hat{\mathbf{x}}) = \theta^{(l)}\hat{\mathbf{x}} \qquad , l = 1$$

$$\mathbf{a}^{(l)}(\hat{\mathbf{h}}^{(l-1)}) = \theta^{(l)}\hat{\mathbf{h}}^{(l-1)} \qquad , l > 1$$

Deep feedforward networks

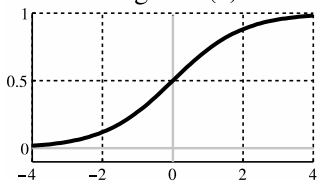
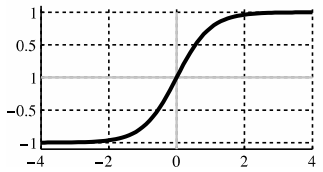
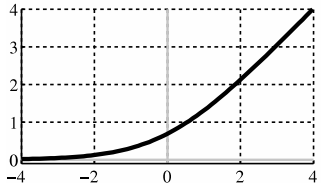
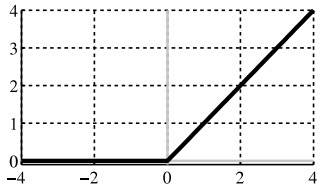


- Unlike Bayesian networks the hidden units here are *intermediate deterministic computations* not random variables, which is why they are not represented as circles
- However, the output variables y_k are drawn as circles because they can be

Activation functions

- Activation functions, h(l)(x) generally operate on the pre-activation vectors in an element-wise fashion
- While sigmoid functions have been popular, the hyperbolic tangent function is sometimes preferred, partly because it has a steady state at 0
- More recently the rectify() function or rectified linear units (ReLUs) have been found to yield superior results in many different settings
  - ✓Since ReLUs are 0 for negative argument values, some units in the model will yield activations that are 0, giving a sparseness property that is useful in many contexts
  - ✓The gradient is particularly simple—either 0 or 1
  - ✓This helps address the exploding gradient problem
- A number of software packages make it easy to use a variety of activation functions, determining gradients automatically using symbolic computations

Activation functions

Name and Graph	Function	Derivative
sigmoid(x) 	$h(x) = \frac{1}{1 + \exp(-x)}$	$h'(x) = h(x)[1 - h(x)]$
tanh(x) 	$h(x) = \frac{\exp(x) - \exp(-x)}{\exp(x) + \exp(-x)}$	$h'(x) = 1 - h(x)^2$
softplus(x) 	$h(x) = \log(1 + \exp(x))$	$h'(x) = \frac{1}{1 + \exp(-x)}$
rectify(x) 	$h(x) = \max(0, x)$	$h'(x) = \begin{cases} 1 \text{ if } x \geq 0 \\ 0 \text{ if } x < 0 \end{cases}$



Dimension Reduction

Dimension Reduction

Ý tưởng: Chuyển đổi dữ liệu từ không gian nhiều chiều sang ít chiều hơn

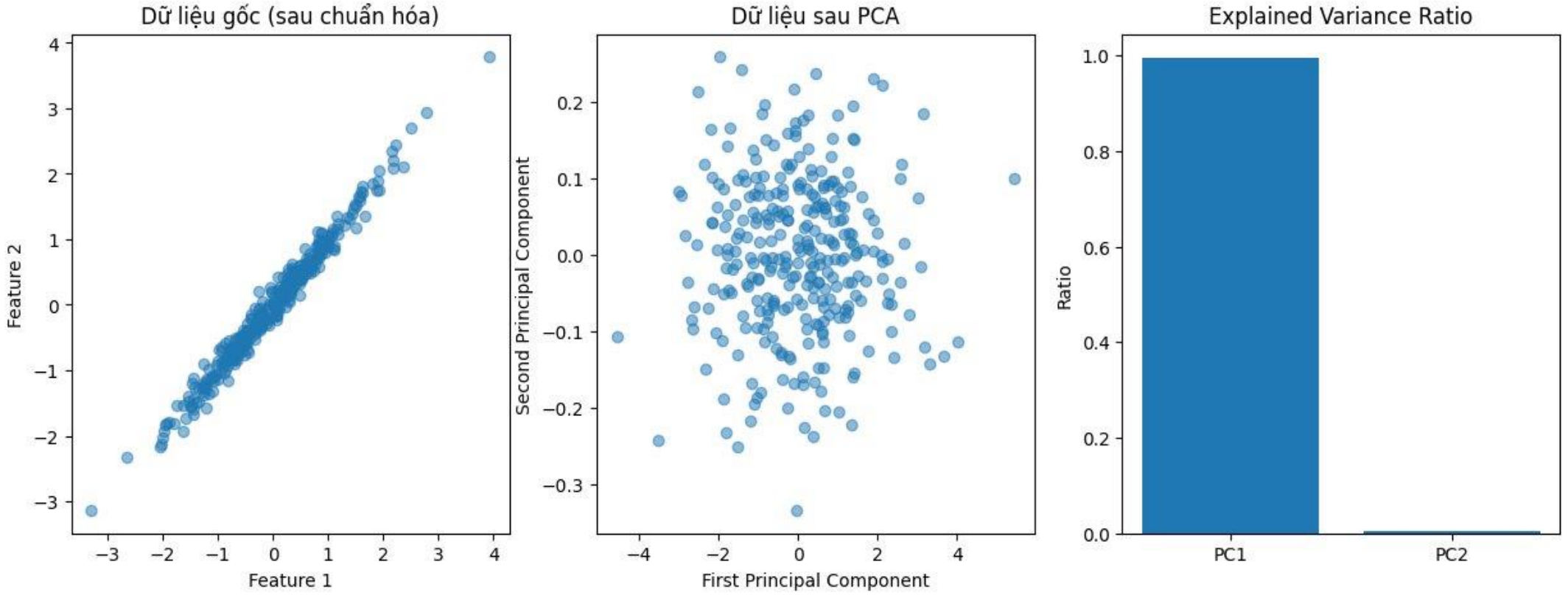
Dimension Reduction

Tại sao cần Dimension Reduction?

- Giảm độ phức tạp của mô hình
- Giảm chi phí tính toán và lưu trữ
- Tránh hiện tượng "curse of dimensionality"
- Loại bỏ nhiễu và dữ liệu dư thừa
- Dễ dàng trực quan hóa dữ liệu

Dimension Reduction

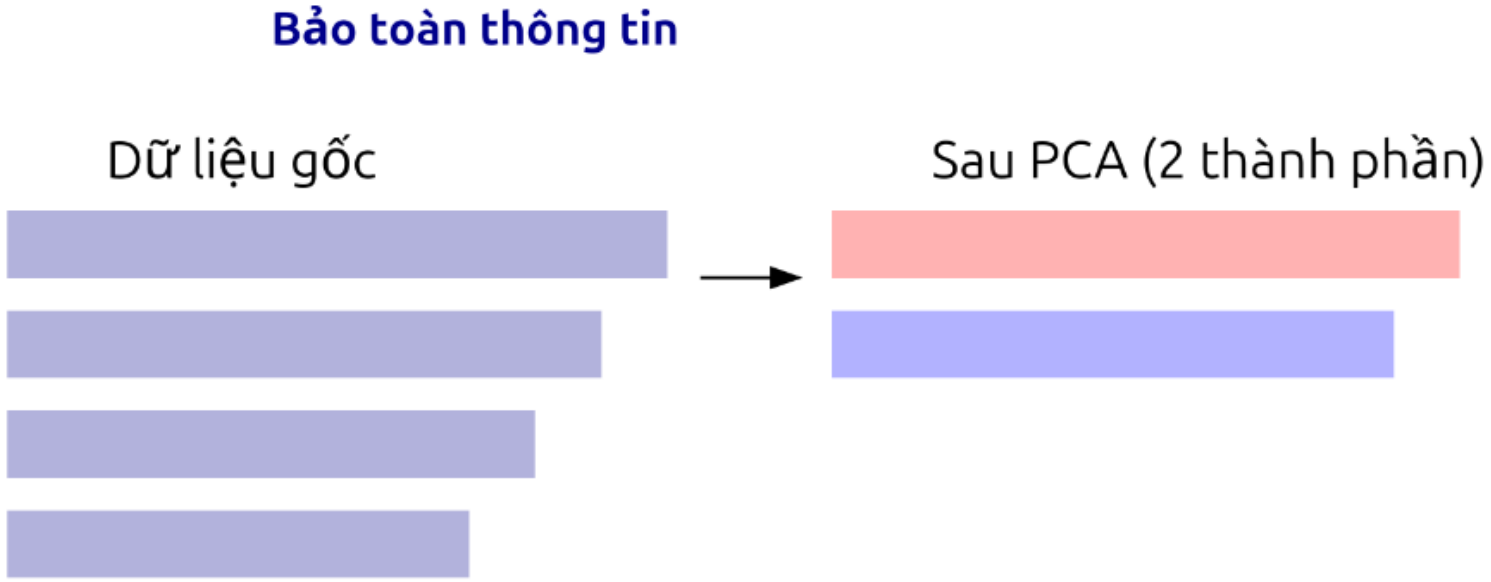
Ví dụ giảm chiều bằng Principal Component Analysis (PCA)



Dimension Reduction

Principal Component Analysis (PCA)

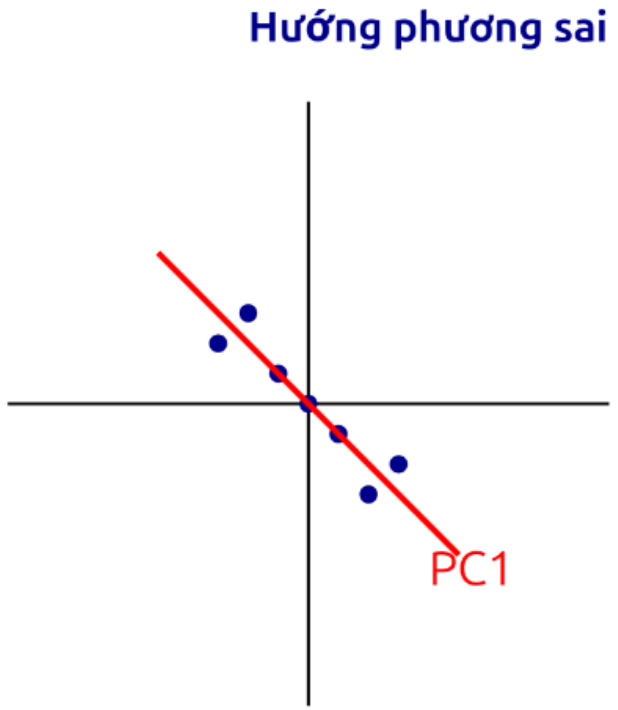
Phương pháp giảm chiều tuyến tính phổ biến nhất

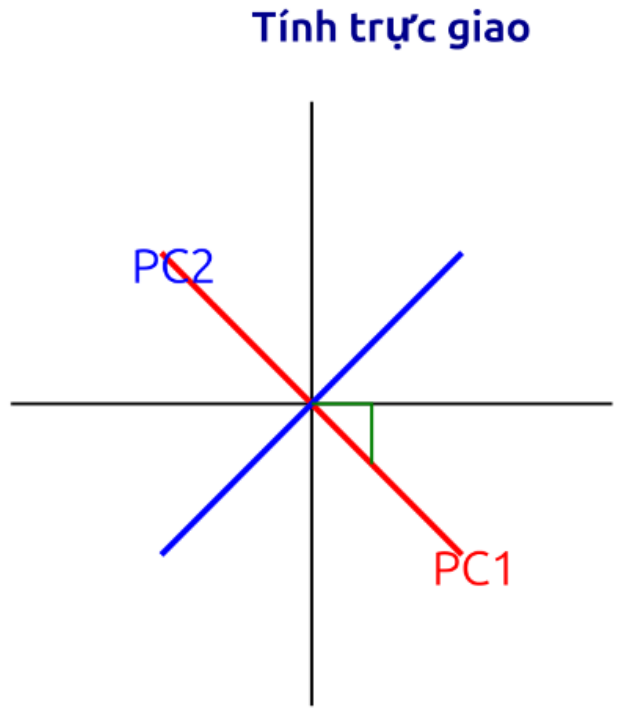


Dimension Reduction

Nguyên lý PCA

- Tìm hướng có **phương sai** lớn nhất
- Trực giao với nhau
- Giữ lại thông tin quan trọng nhất





Dimension Reduction

Các bước thực hiện PCA

1. Chuẩn hóa dữ liệu
2. Tính ma trận hiệp phương sai
3. Tính vector riêng và giá trị riêng
4. Sắp xếp và chọn components

Dimension Reduction

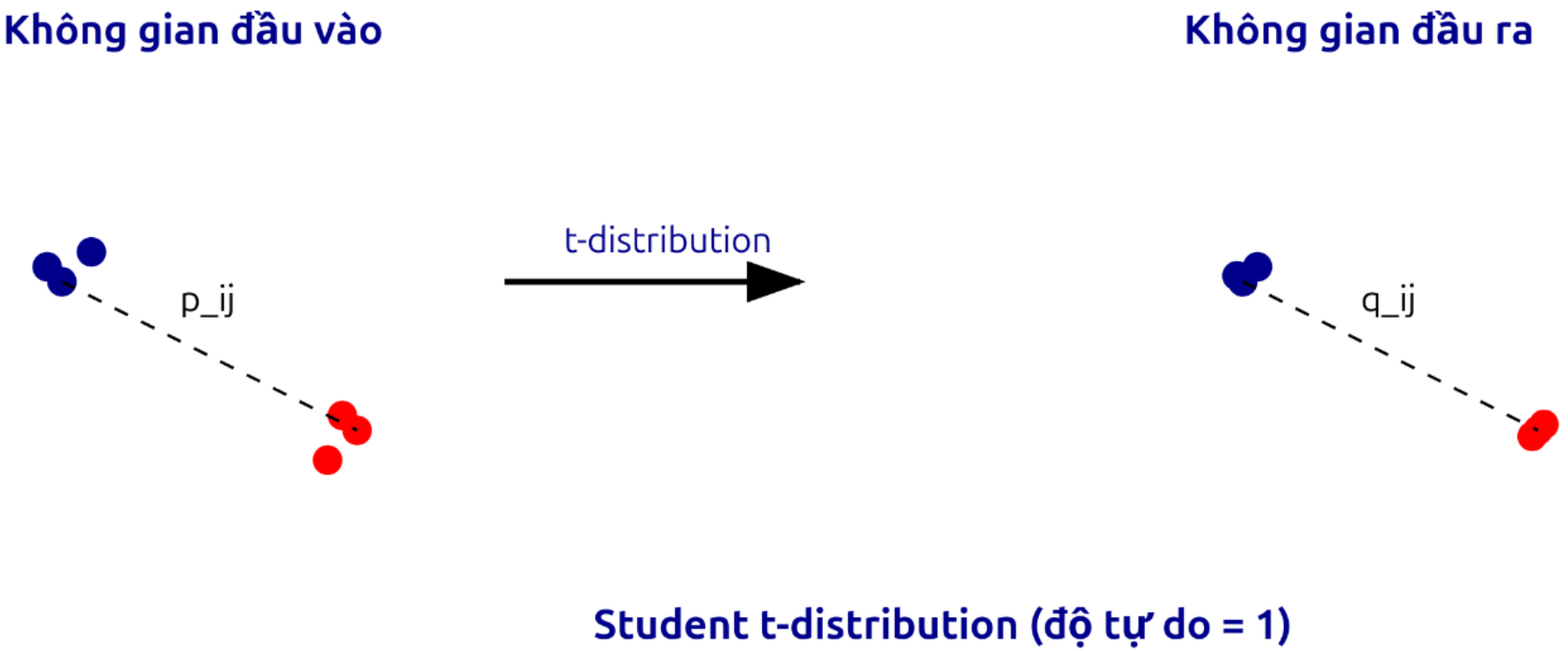
Ưu điểm PCA

- Đơn giản, hiệu quả
- Giảm nhiễu
- Không mất nhiều tham số
- Dễ thực hiện

Nhược điểm PCA

- Chỉ hoạt động với quan hệ tuyến tính
- Khó giải thích các components
- Nhạy cảm với outliers

t-SNE


Không gian đầu vào
Không gian đầu ra
p_ij
t-distribution
q_ij
Student t-distribution (độ tự do = 1)

Dimension Reduction

Nguyên lý t-SNE

- Bảo toàn cấu trúc cục bộ
- Dựa trên xác suất có điều kiện
- Sử dụng Student t-distribution

Dimension Reduction

Ưu điểm t-SNE

- Hiệu quả với dữ liệu phi tuyến
- Bảo toàn cấu trúc cục bộ tốt
- Trực quan hóa hiệu quả

Nhược điểm t-SNE

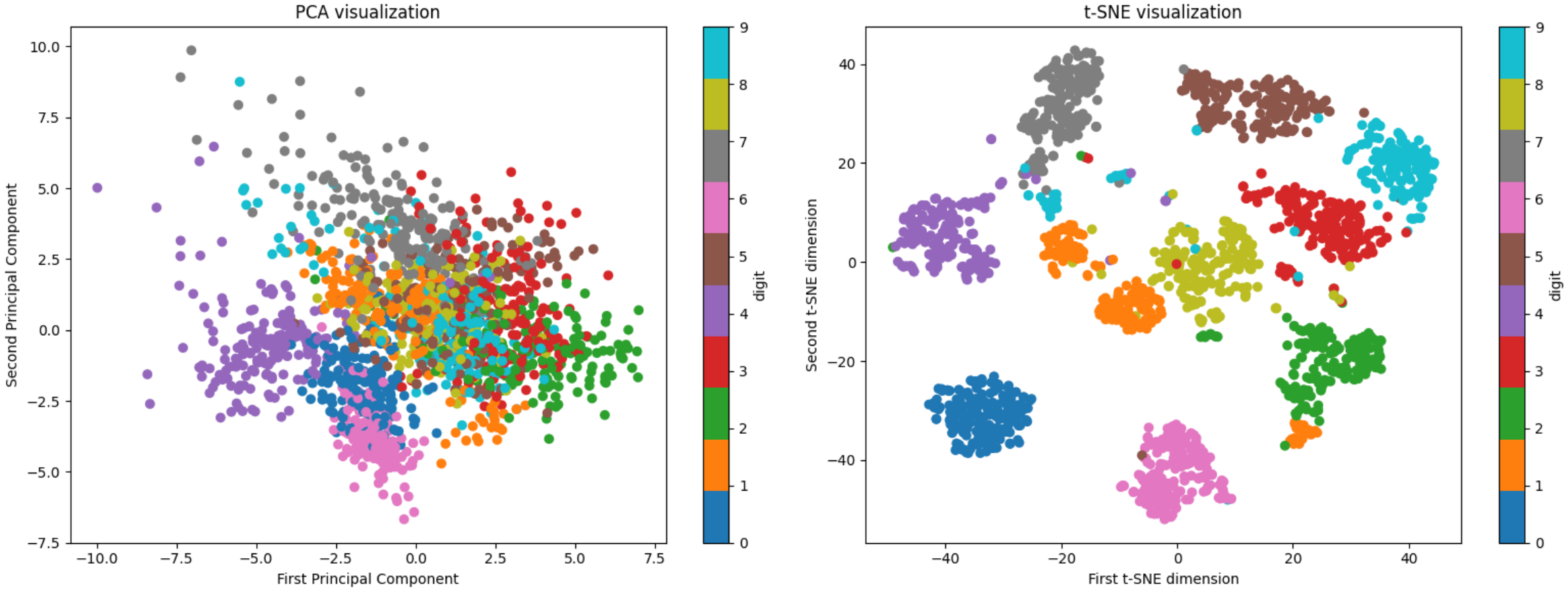
- Chậm với dữ liệu lớn
- Không bảo toàn khoảng cách toàn cục
- Kết quả phụ thuộc perplexity

Dimension Reduction

So sánh PCA và t-SNE ???

Dimension Reduction

Bài tập: Ứng dụng thực tế trên bộ dữ liệu **MNIST** digits





Feature Engineering

Bộ môn Khoa học máy tính - Khoa Hệ thống thông tin quản lý

Feature Engineering

Feature Engineering là quá trình tạo ra **các đặc trưng mới** từ dữ liệu thô ban đầu, nhằm cải thiện hiệu suất của mô hình học máy - machine learning.

Feature Engineering

Numerical Features

- Scaling/Normalization
- Log transformation
- Polynomial features
- Binning/Discretization

Feature Engineering

Categorical Features

- **One-hot encoding**
- Label encoding
- Target encoding
- **Feature hashing**

Feature Engineering

Text Features

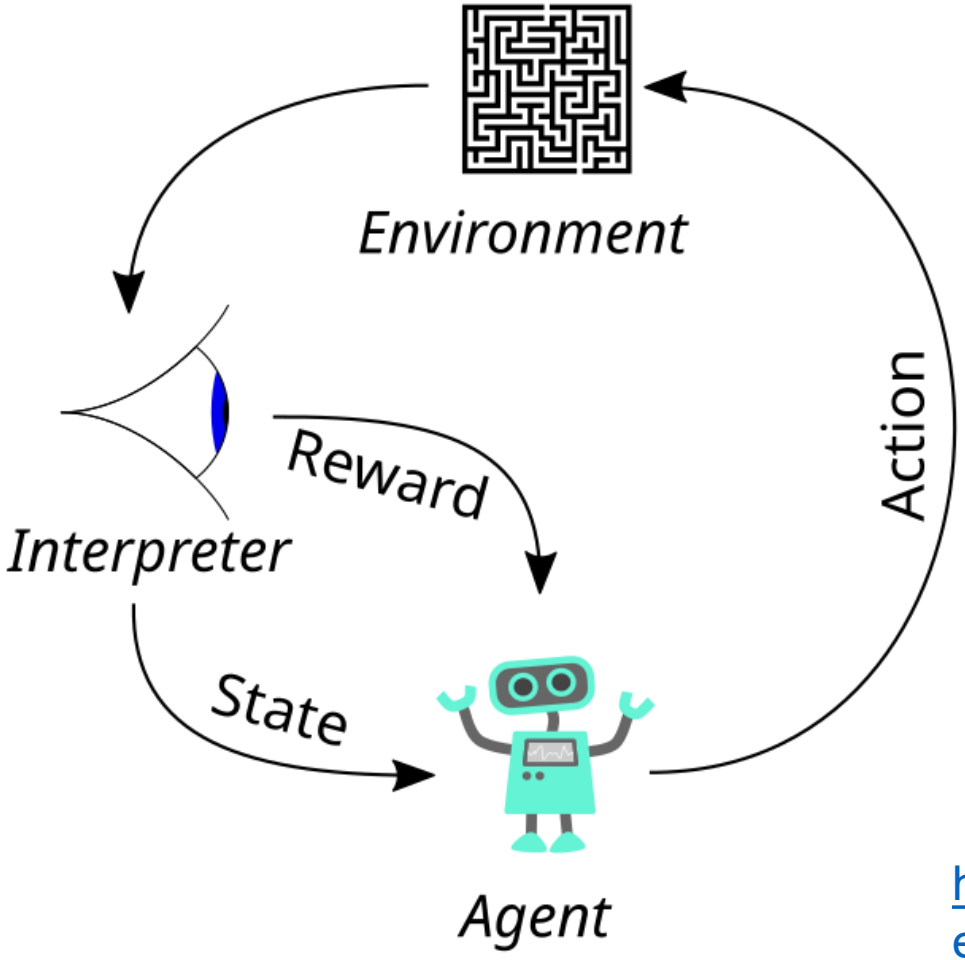
- Bag of words
- TF-IDF
- N-grams
- **Word embeddings**

Feature Engineering

Time Features

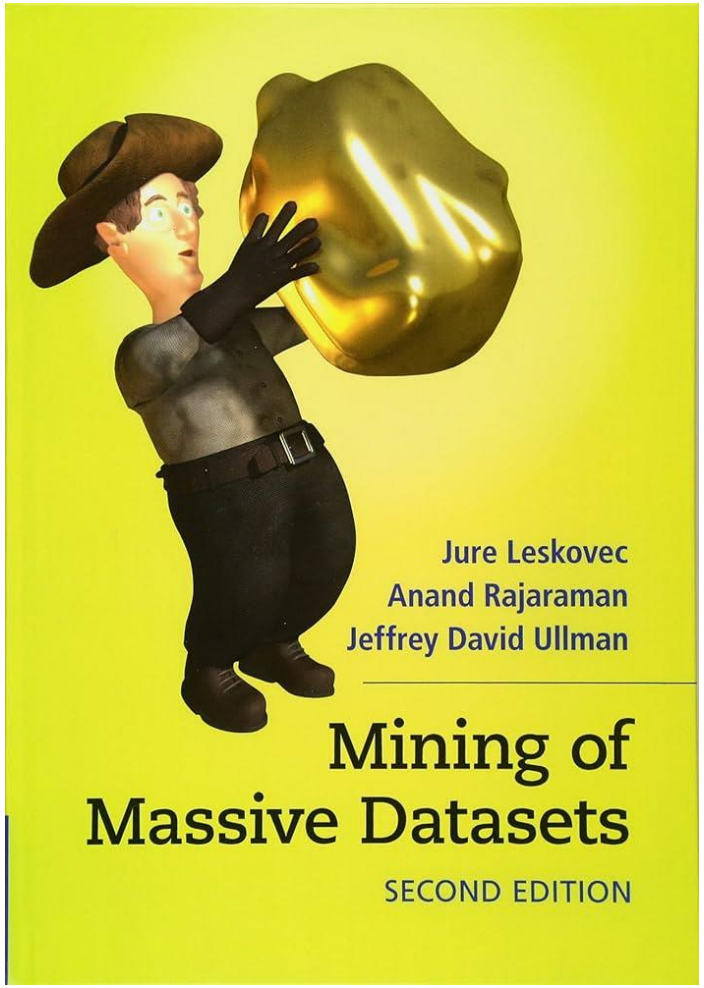
- Extracting datetime components
- Time-based aggregations
- **Lag features**
- Rolling statistics

Reinforcement Learning



https://en.wikipedia.org/wiki/Reinforcement_learning

Mining of Massive Datasets

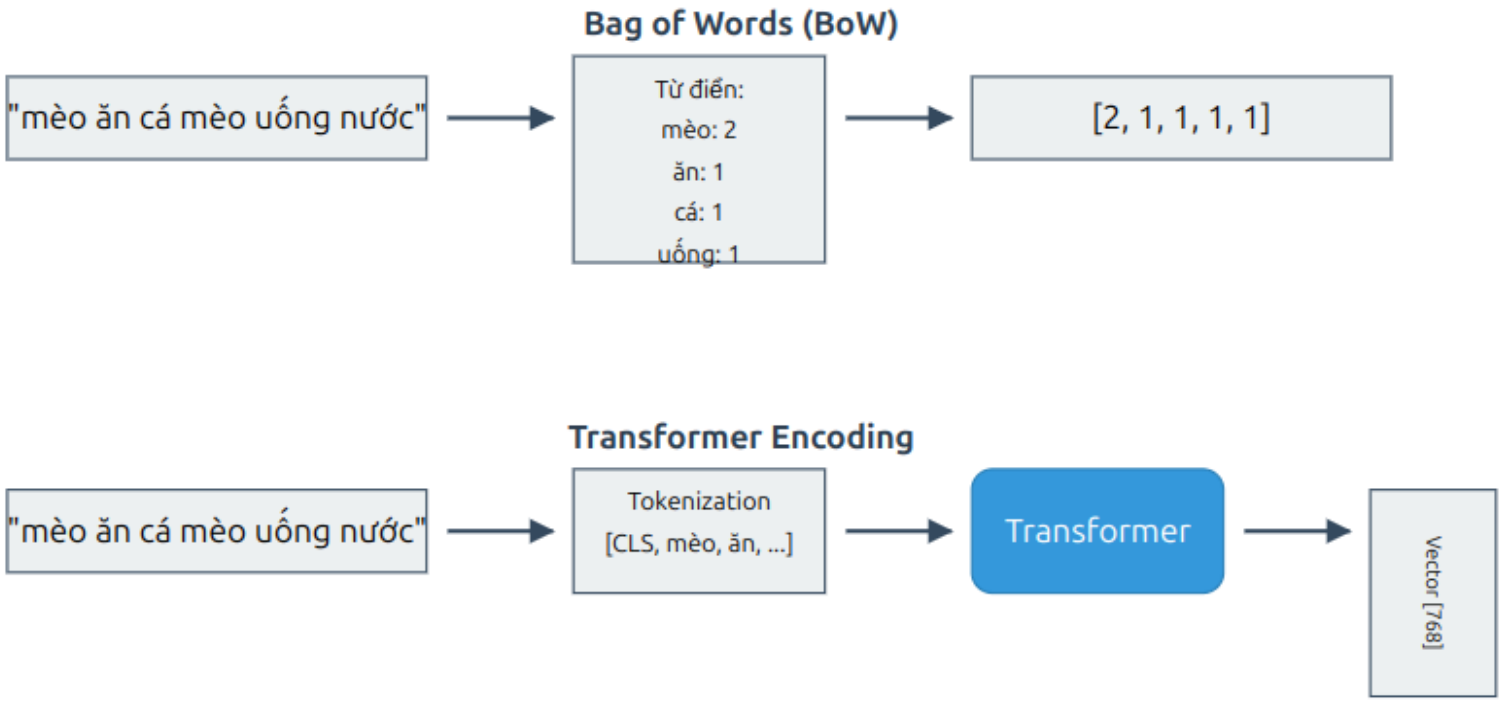


Large Language Model & Generative AI

- Mô hình truyền thống Bag of Word
- Mô hình ngôn ngữ lớn Transformer

Large Language Model & Generative AI

So sánh Bag of Words vs Transformer Encoding



Khác biệt chính:

- BoW: Đơn giản, chỉ đếm tần suất từ, không capture được ngữ cảnh và thứ tự
- Transformer: Phức tạp hơn, hiểu được ngữ cảnh, tạo vector có tính semantic



THANK YOU FOR WATCHING!